COMP719 SEMESTER PROJECT
THE CAMPUS LAB SPACE OPTIMIZER
PHASE 3 – FINAL SOLUTION AND ANALYSIS

1. GROUP INFORMATION

Member Name	Student ID	Role	% Contribution
Jeryn Naidoo	223009245	Quality Assurance & Report Coordination	20
Kirone Gopaul	223014456	Developer & Code Architecture	20
Shaista Naicker	222106172	Algorithms & Heuristic Design	20
Josh Goodwin	222092144	Evaluator & Performance Analysis	20
Tashmika Pillay	223097300	Mathematical Modeller & System Design	20

2. EXECUTIVE SUMMARY

This report presents the final solution for the Campus Lab Space Optimizer. The goal of the system is to assign each of ten university modules to one of four computer labs such that every student has a seat, software requirements are met, and the total number of unused seats across all assignments is minimized.

The solution was developed in three stages. In Phase 1, the problem was modelled mathematically and the data was loaded into Pandas DataFrames. In Phase 2, two greedy algorithms were implemented as baseline solutions: Largest Module First (Best-Fit Decreasing) and Smallest Lab First. Both produced the same total of 140 unused seats, satisfying all constraints.

In this final phase, two local search improvement methods were applied on top of each greedy solution: hill-climbing reassignment search and swap search. The hill-climbing method iterates over all modules and reassigns any module to a better-fitting lab if one is found. The swap search considers every pair of modules and exchanges their lab assignments if the swap reduces total waste while keeping both assignments valid.

After applying both improvement methods to both greedy starting points, no further improvements were found, the greedy solutions were already at a local optimum. The final total unused seats for all

pipelines remains 140. All constraints are fully satisfied: every module is assigned to exactly one lab, no lab is over capacity, and all software requirements are met.

3. PROBLEM OVERVIEW

3.1 Problem Description

The University operates four computer laboratories of varying sizes and software configurations. Ten different modules each require access to a computer lab during the semester. Each module has a fixed number of enrolled students and a specific software requirement.

Two operational problems motivated this project: overcrowding occurs when a module is assigned to a lab that is too small for its class size, and underutilization occurs when a small class is assigned to a very large lab, leaving many seats empty. Our system addresses both by finding the assignment that satisfies all hard constraints while minimizing total seat wastage.

A key assumption is that all modules are scheduled at different times, meaning a single lab can host multiple modules across the semester, but each module must be assigned to exactly one lab.

3.2 Objectives and Constraints

Objective Function:

Minimize the total number of unused seats summed across all module-lab assignments.

Key Constraints:

- Unique Assignment: Each module must be assigned to exactly one lab.
- Capacity Constraint: A module may only be assigned to a lab whose capacity is greater than or equal to the module's student count.
- Software Compatibility: A module may only be assigned to a lab that has the required software installed.

4. FINAL MATHEMATICAL MODEL

4.1 Decision Variables

Let $x_{ij} = 1$ if module i is assigned to lab j, and 0 otherwise, where:

- $i \in \{M1, M2, M3, M4, M5, M6, M7, M8, M9, M10\}$ (the set of all modules)
- $j \in \{L1, L2, L3, L4\}$ (the set of all labs)

4.2 Final Constraints

Unique Assignment Constraint:

$\Sigma_j x_{ij} = 1$ for all i

Each module must be assigned to exactly one lab. No module may be left unassigned or assigned to more than one lab.

Capacity Constraint:

$x_{ij} = 0$ if Students(i) > Capacity(j)

A module can only be placed in a lab that has enough seats for all enrolled students.

Software Compatibility Constraint:

$x_{ij} = 0$ if SoftwareNeeded(i) $\notin$ Software(j)

A module can only be placed in a lab where its required software is installed.

4.3 Objective Function

Minimize: $\Sigma_i \Sigma_j x_{ij} \times (\text{Capacity}(j) - \text{Students}(i))$

For each valid assignment, unused seats = Lab Capacity − Number of Students. The objective minimizes the sum of these values across all assigned pairs.

5. FINAL SOLUTION APPROACH

5.1 Greedy Algorithm (Baseline Solution)

Two greedy algorithms were implemented to produce an initial feasible solution:

- **Greedy Algorithm 1 - Largest Module First (Best-Fit Decreasing):**

  Modules are sorted in descending order of student count. For each module, every lab is evaluated and the one that produces the least waste (smallest compatible lab that still fits) is selected. The rationale is that larger modules have fewer compatible labs, so handling them first prevents a situation where a large module has no valid lab remaining.

- **Greedy Algorithm 2 - Smallest Lab First:**

  Modules are processed in their original order (M1 through M10). Labs are sorted from smallest to largest capacity, and the first lab that satisfies both constraints is selected. This minimizes waste at each individual step without regard to global ordering.

Both algorithms were chosen for their simplicity, their guaranteed feasibility, and their role as effective starting points for local search improvement.

5.2 Improvement Method (Local Search or Similar)

Two local search methods were applied sequentially to improve each greedy solution:

- **Hill-Climbing Reassignment Search:**

  This method iterates over every module in the current assignment and attempts to find a compatible lab with a smaller capacity gap. If an improvement is found, the module is reassigned and the search restarts. This continues until a full pass through all modules yields no improvement, indicating a local optimum has been reached.

- **Swap Search:**

  This method considers every pair of modules and evaluates whether swapping their lab assignments would reduce the combined unused seats. A swap is only accepted if both modules remain in valid labs after the exchange and the total waste decreases. This targets improvements that single-module reassignment cannot find, because moving module A into a better lab might first require moving module B out.

Hill climbing was chosen because it is intuitive and directly targets the objective. Swap search complements it by exploring a wider neighbourhood that single-move reassignment miss.

5.3 Neighbourhood Definition

A neighbourhood move is any small change to the current assignment that produces a new valid assignment. Two neighbourhood types are defined:

- **Reassignment move:** A single module is moved from its current lab to a different compatible lab. The move is accepted only if the new lab reduces unused seats for that module.
- **Swap move:** Two modules exchange their lab assignments. The move is accepted only if both remain in valid labs and the combined unused seats decrease.

6. Implementation Details

6.1 System Architecture

The initial data setup begins by defining two datasets: one containing capacity and software availability, and another containing information about individual classes. Compatibility checks are then performed to determine which modules can fit into which labs.

```
def check_fit(module_index, lab_index):

    module = df_modules.iloc[module_index]
    lab = df_labs.iloc[lab_index]

    has_space = module['Students'] <= lab['Capacity']
    has_software = module['Software Needed'] in lab['Software']

    return has_software and has_space
```

Data visualization is then performed on the compatible labs.

When creating a baseline with a greedy approach to this problem, the largest-first and smallest-first algorithms were used to investigate the efficacy of a greedy algorithm for this specific problem. The following block of code is for the greedy largest-first algorithm. A similar method is used for the greedy smallest-first algorithm.

```
def greedy_largest_first():
    # Sort the modules so the biggest ones (most students) go first
    sorted_modules = df_modules.sort_values('Students', ascending=False)
    sorted_modules = sorted_modules.reset_index(drop=True)

    result_rows = []

    # Go through every module one at a time
    for i in range(len(sorted_modules)):
        module = sorted_modules.iloc[i]

        best_lab = None
        least_waste = 999999  # Start with a big number so any real waste will
be smaller

        # Check every lab to find the one that wastes the fewest seats
        for j in range(len(df_labs)):
            lab = df_labs.iloc[j]

            # Does this lab have the right software?
            software_ok = module['Software Needed'] in lab['Software']

            # Does this lab have enough seats?
            space_ok = module['Students'] <= lab['Capacity']

            if software_ok and space_ok:
                wasted_seats = lab['Capacity'] - module['Students']

                # If this lab wastes fewer seats than the current best, use it
instead
```

```
                if wasted_seats < least_waste:
                    least_waste = wasted_seats
                    best_lab = lab

        if best_lab is None:
          raise ValueError(f"No feasible lab found for module {module['Module
Code']}")

        # Save the result for this module
        row = {
            'Module Code': module['Module Code'],
            'Students': module['Students'],
            'Lab ID': best_lab['Lab ID'],
            'Capacity': best_lab['Capacity'],
            'Unused Seats': least_waste
        }
        result_rows.append(row)

    return pd.DataFrame(result_rows)
```

After evaluating both algorithms, both are displayed and compared to each other using a bar chart.

Finally, heuristic algorithms are introduced to compare a greedy approach to a heuristic approach. Another two algorithms, hill-climb and swap search, are used to find a solution to the problem, and compare the heuristic and greedy approaches.

```
def hill_climb(df_assignment):
    # Build a dict so we can look up and update assignments easily
    assignment = {}
    for i in range(len(df_assignment)):
        row = df_assignment.iloc[i]
        assignment[row['Module Code']] = row['Lab ID']

    log_rows = []
    improved = True

    while improved:
        improved = False

        for i in range(len(df_modules)):
            module = df_modules.iloc[i]
            mod_code = module['Module Code']
            current_lab_id = assignment[mod_code]

            # Find the current lab row
            current_lab = df_labs[df_labs['Lab ID'] == current_lab_id].iloc[0]
            current_waste = current_lab['Capacity'] - module['Students']
```

```
        # Try every other compatible lab to see if we can do better
        for j in range(len(df_labs)):
            lab = df_labs.iloc[j]

            if lab['Lab ID'] == current_lab_id:
                continue

            software_ok = module['Software Needed'] in lab['Software']
            space_ok = module['Students'] <= lab['Capacity']

            if software_ok and space_ok:
                new_waste = lab['Capacity'] - module['Students']

                if new_waste < current_waste:
                    log_rows.append({
                        'Module Code': mod_code,
                        'From Lab': current_lab_id,
                        'To Lab': lab['Lab ID'],
                        'Waste Before': current_waste,
                        'Waste After': new_waste,
                        'Gain': current_waste - new_waste
                    })
                    assignment[mod_code] = lab['Lab ID']
                    improved = True
                    break

        if improved:
            break

# Rebuild result DataFrame
result_rows = []
for i in range(len(df_modules)):
    module = df_modules.iloc[i]
    mod_code = module['Module Code']
    lab_id = assignment[mod_code]
    lab = df_labs[df_labs['Lab ID'] == lab_id].iloc[0]
    result_rows.append({
        'Module Code': mod_code,
        'Students': module['Students'],
        'Lab ID': lab_id,
        'Capacity': lab['Capacity'],
        'Unused Seats': lab['Capacity'] - module['Students']
    })

return pd.DataFrame(result_rows), pd.DataFrame(log_rows)
```

```
def swap_search(df_assignment):
    # Build a working dict from the DataFrame
    assignment = {}
    for i in range(len(df_assignment)):
        row = df_assignment.iloc[i]
        assignment[row['Module Code']] = row['Lab ID']

    module_codes = list(assignment.keys())
    log_rows = []
    improved = True

    while improved:
        improved = False

        # Try every pair of modules and see if swapping their labs helps
        for i in range(len(module_codes)):
            for j in range(i + 1, len(module_codes)):
                mod_a = module_codes[i]
                mod_b = module_codes[j]
                lab_a = assignment[mod_a]
                lab_b = assignment[mod_b]

                if lab_a == lab_b:
                    continue

                module_a = df_modules[df_modules['Module Code'] ==
mod_a].iloc[0]
                module_b = df_modules[df_modules['Module Code'] ==
mod_b].iloc[0]
                lab_row_a = df_labs[df_labs['Lab ID'] == lab_a].iloc[0]
                lab_row_b = df_labs[df_labs['Lab ID'] == lab_b].iloc[0]

                # Check if the swap is feasible for both modules
                a_fits_b = module_a['Software Needed'] in lab_row_b['Software']
and module_a['Students'] <= lab_row_b['Capacity']
                b_fits_a = module_b['Software Needed'] in lab_row_a['Software']
and module_b['Students'] <= lab_row_a['Capacity']

                if a_fits_b and b_fits_a:
                    waste_before = (lab_row_a['Capacity'] -
module_a['Students']) + (lab_row_b['Capacity'] - module_b['Students'])
                    waste_after  = (lab_row_b['Capacity'] -
module_a['Students']) + (lab_row_a['Capacity'] - module_b['Students'])

                    if waste_after < waste_before:
```

```
                        log_rows.append({
                            'Swap': f"{mod_a} ↔ {mod_b}",
                            'Labs': f"{lab_a} ↔ {lab_b}",
                            'Waste Before': waste_before,
                            'Waste After': waste_after,
                            'Gain': waste_before - waste_after
                        })
                        assignment[mod_a] = lab_b
                        assignment[mod_b] = lab_a
                        improved = True
                        break

            if improved:
                break

    # Rebuild result DataFrame
    result_rows = []
    for i in range(len(df_modules)):
        module = df_modules.iloc[i]
        mod_code = module['Module Code']
        lab_id = assignment[mod_code]
        lab = df_labs[df_labs['Lab ID'] == lab_id].iloc[0]
        result_rows.append({
            'Module Code': mod_code,
            'Students': module['Students'],
            'Lab ID': lab_id,
            'Capacity': lab['Capacity'],
            'Unused Seats': lab['Capacity'] - module['Students']
        })
      return pd.DataFrame(result_rows), pd.DataFrame(log_rows)
```

Following the two heuristics, a final comparison of all the algorithms is displayed using a bar graph.

6.2 Key Functions

Function Name	Purpose
`check_fit(module_index, lab_index)`	Returns True if the module fits in the lab (capacity and software checks). Used as the feasibility gate for all algorithms.
`display_compatability_info()`	Builds a list of compatible lab IDs for each module and stores it in the DataFrame.

`greedy_largest_first()`	Sorts modules by student count descending, then assigns each to the least-wasteful compatible lab.
`greedy_smallest_lab_first()`	Processes modules in original order, assigning each to the smallest compatible lab.
`hill_climb(df_assignment)`	Applies hill-climbing reassignment search to a given assignment DataFrame. Returns the improved assignment and a log of changes.
`swap_search(df_assignment)`	Applies pairwise swap search to a given assignment DataFrame. Returns the improved assignment and a log of accepted swaps.

6.3 Data Structures

- Lab data: Stored as a Pandas DataFrame (df_labs) with columns Lab ID, Name, Capacity, and Software (list).
- Module data: Stored as a Pandas DataFrame (df_modules) with columns Module Code, Students, Software Needed, and Compatible Labs (list, added after compatibility mapping).
- Assignment results: Each algorithm returns a DataFrame with columns Module Code, Students, Lab ID, Capacity, and Unused Seats.
- Hill-climb and swap logs: Each improvement function also returns a log DataFrame recording each accepted move, the waste before and after, and the gain.
- Working assignment state: Inside hill_climb() and swap_search(), assignments are held in a Python dictionary keyed by Module Code for O(1) lookup and update.

7. RESULTS AND EVALUATION

7.1 Greedy Solution Results

Greedy Algorithm 1 - Largest Module First:

	Module Code	Students	Lab ID	Capacity	Unused Seats
0	M7	90	L1	100	10
1	M1	85	L1	100	15
2	M4	55	L2	60	5
3	M10	50	L2	60	10
4	M2	45	L1	100	55

	Module Code	Students	Lab ID	Capacity	Unused Seats
5	M5	35	L3	40	5
6	M8	30	L3	40	10
7	M3	25	L4	30	5
8	M9	20	L4	30	10
9	M6	15	L4	30	15

Total Unused Seats (Greedy 1): 140

Greedy Algorithm 2 - Smallest Lab First:

	Module Code	Students	Lab ID	Capacity	Unused Seats
0	M1	85	L1	100	15
1	M2	45	L1	100	55
2	M3	25	L4	30	5
3	M4	55	L2	60	5
4	M5	35	L3	40	5
5	M6	15	L4	30	15
6	M7	90	L1	100	10
7	M8	30	L3	40	10
8	M9	20	L4	30	10
9	M10	50	L2	60	10

Total Unused Seats (Greedy 2): 140

7.2 Improved Solution Results

Heuristic Algorithm 1 – Hill Climbing (Same Results for Largest Module First & Smallest Lab First):

	Module Code	Students	Lab ID	Capacity	Unused Seats
0	M1	85	L1	100	15
1	M2	45	L1	100	55
2	M3	25	L4	30	5
3	M4	55	L2	60	5
4	M5	35	L3	40	5
5	M6	15	L4	30	15
6	M7	90	L1	100	10
7	M8	30	L3	40	10
8	M9	20	L4	30	10
9	M10	50	L2	60	10

```
No improvements found — greedy solution is already at a local optimum.
```

Total Unused Seats (Heuristic 1): 140

Heuristic Algorithm 2 – Swap Search (Same Results for Largest Module First & Smallest Lab First):

	Module Code	Students	Lab ID	Capacity	Unused Seats
0	M1	85	L1	100	15
1	M2	45	L1	100	55
2	M3	25	L4	30	5
3	M4	55	L2	60	5
4	M5	35	L3	40	5
5	M6	15	L4	30	15
6	M7	90	L1	100	10

	Module Code	Students	Lab ID	Capacity	Unused Seats
7	M8	30	L3	40	10
8	M9	20	L4	30	10
9	M10	50	L2	60	10

```
No improving swaps found — solution confirmed at local optimum.
```

Total Unused Seats (Heuristic 2): 140

7.3 Comparison of Greedy vs Improved solution

Metric	Greedy	Final Improved
Total Unused Seats	140	140
Valid Solution	Yes	Yes

8. SOLUTION VALIDITY

Every module is assigned exactly once:

Both greedy algorithms iterate over all ten modules exactly once and append exactly one row per module to the result DataFrame. No module is skipped and no module appears twice.

Capacity constraints are never violated:

Every assignment decision passes through the check_fit() function, which enforces the condition module['Students'] <= lab['Capacity'] before any lab is accepted. Labs that are too small are structurally excluded from consideration.

Software compatibility is always satisfied:

check_fit() also enforces module['Software Needed'] in lab['Software']. A lab is only selected if its installed software list includes the module's required software.

The hill-climbing and swap search functions preserve feasibility by applying the same checks before accepting any move or swap. It is impossible for the improvement phase to produce an infeasible assignment.

9. PERFORMANCE DISCUSSION

9.1 What Worked Well

The Largest Module First strategy was very effective on this dataset. We sorted the modules in descending order of class size, and assigned each class to the smallest compatible lab, which minimized the wasted space at every step. The outcome of 140 total unused seats across the 10 modules was confirmed by both the hill-climbing, and swap-search phases to be optimal, and no further improvements were found.

The local search implementation was equally effective in terms of correctness. Both the hill-climbing reassignment search, and the pairwise swap search evaluated all neighbours correctly, respecting all constraints, and only accepting improvements when waste was genuinely reduced. The code was designed modularly, with separate functions for each phase which made independent testing of the code easy and convenient.

We also used bar graphs to compare the algorithms and track progress across all of the stages, which provided clear and easy to follow confirmation that no degradation occurred between the stages.

9.2 Limitations

The greedy algorithm struggles to efficiently explore the full solution space due to the small dataset (only four labs and ten modules), which means that the improvement phases had little opportunity to demonstrate their value. If the dataset were larger and more complex (dozens of labs and hundreds of modules), the greedy solution would not be likely to reach a local optimum immediately, so the improvement methods would play a more significant role.

The second limitation is that the current model only cares about the capacity and available software of each lab, whereas in a real-world scenario, there would be additional constraints to consider, such as timetable conflicts, and possibly hardware-specific needs of each module.

Finally, the hill-climbing implementation uses a first-improvement restart strategy, where the outer loop is restarted as soon as one improvement is found. While this does work and is efficient for this problem size, it may miss improvements on larger instances where multiple non-conflicting moves exist simultaneously.

9.3 Possible Future Improvements

- Apply more advanced metaheuristics such as simulated annealing or a genetic algorithm which can escape local optima and explore a broader solution space on a larger dataset
- Incorporate time-slot scheduling as an additional constraint, simulating the real-world application of lab scheduling more accurately
- Extending the objective function to include other criteria such as minimizing the number of labs used to reduce energy consumption, or to balance lab utilization

- Implement a multi-start strategy where the greedy algorithm and improvement phases are run multiple times with different initial orderings, and the best result is taken, improving robustness

10. CHALLENGES AND LESSONS LEARNED

Typical technical challenges faced were the local optimum problem, order sensitivity, and combinatorial explosion.

Considering heuristic algorithms like Hill-Climbing and Swap Search, both risk getting stuck at local optima. This means if they reach a solution and see no improvement in the immediate local neighbourhood, they stop, even if a better solution exists further in the search space.

The order sensitivity problem occurs because the final solution is dependent on the starting point. Because all the algorithms used are local search algorithms, they do not look ahead. If a seemingly good solution occurs early in the search, it may block a better solution from being found

A combinatorial explosion may occur using the solutions proposed in this project. This is because this code only handles 10 modules. If a real-world scenario with more modules needed to be optimized, the current algorithms would likely lead to the number of possible combinations growing exponentially.

The primary modelling challenge faced was that the proposed solutions do not scale well to real-world problems. A real-world problem will typically have multiple objectives that need to be optimized, balancing all possible constraints and objectives.

What we learnt:

- **Jeryn Naidoo:**
  Taking on the QA role taught me that checking work is a lot more involved than I expected. It's not just running the code and seeing if it produces output; it's verifying that every result is actually correct, that the tables match the code, that the constraints are genuinely satisfied and not just assumed to be. Coordinating the report also showed me how much can get lost when different sections are written separately. Getting everything to read consistently and flow as one document took more effort than the writing itself.

- **Shaista Naicker:**
  Designing the greedy algorithms and the local search methods made me realise how much thought goes into what seems like a simple rule. Deciding to process larger modules first wasn't obvious at first, it only made sense once I thought carefully about which modules have the fewest compatible labs and would therefore cause the most problems if left until last. The heuristic design process is less about finding the cleverest trick and more about understanding the structure of your problem well enough to make informed decisions at every step.

- **Kirone Gopaul:**

  Implementing the hill-climbing and swap search functions helped me bridge the gap between the theoretical concept of neighbourhood search and what it actually looks like in code. The more valuable learning came from debugging, subtle issues like failing to restart the search after an accepted move revealed how sensitive these algorithms are to implementation details. It also gave me a more realistic picture of what local search can and can't do, and why more advanced techniques like simulated annealing exist in the first place.

- **Josh Goodwin:**

  Doing the evaluation made me think more critically about what a result actually means. Both greedy algorithms scored 140 unused seats, but through completely different assignments, and deciding whether one was better than the other when the numbers were identical was genuinely difficult. It taught me that performance analysis isn't just reporting numbers, it's asking the right questions about what those numbers do and don't tell you. A single metric rarely captures the full picture. It also made clear that local search is only as useful as the gap between your starting solution and the optimum allows.

- **Tashmika Pillay:**

  Building the mathematical model was the part of this project that surprised me most. I came in thinking it would just be translating the problem into some notation, but it actually required making real decisions on what counts as a variable, what counts as a constraint, how to express the objective in a way that the algorithm can actually optimise. Getting that foundation right mattered more than I expected, because every algorithm built on top of it depended on those definitions being precise. If the model is vague, the solution is vague too.

11. TESTING AND VALIDATION:

Testing Edge Cases: Tests were performed to ensure that the hill-climbing and swap-search functions correctly handle cases where no improvements exist. When both algorithms were applied to the greedy output, the log DataFrames that were returned were empty, and the total unused seats remained at 140, which matches the greedy baseline exactly. A message indicating that no improvements were found, and the solution is already at a local optimum was correctly displayed.

Infeasible Assignment Testing: The check_fit() function was tested against known infeasible combinations to confirm that they were correctly rejected:

- M1 assigned to L2: 85 students assigned to 60 seats, correctly rejected due to violating capacity constraint
- M2 assigned to L2: Students needing MATLAB assigned to lab with Java only, correctly rejected due to violating software constraint
- M7 assigned to L2: 90 students assigned to 60 seats, correctly rejected due to violating capacity constraint

In all cases, the greedy function raised an error if no feasible lab could be found. This prevents any invalid assignments from entering the solution.

Constraint Satisfaction Testing: Validity checks were performed on every module-lab pair, where two conditions were verified:

- Capacity: has_space = module['Students'] <= lab['Capacity']
- Software: has_software = module['Software Needed'] in lab['Software']

Unique Assignment Check: The final assignment DataFrame was checked to confirm it contained exactly ten rows with no duplicate module codes, verifying that every module was assigned exactly once and none were skipped.

Module	Students	Lab ID	Capacity	Unused Seats	Capacity Constraint Satisfied?	Software Constraint Satisfied?
M1	85	L1	100	15	✓	✓
M2	45	L1	100	55	✓	✓
M3	25	L4	30	5	✓	✓
M4	55	L2	60	5	✓	✓
M5	35	L3	40	5	✓	✓
M6	15	L4	30	15	✓	✓
M7	90	L1	100	10	✓	✓
M8	30	L3	40	10	✓	✓
M9	20	L4	30	10	✓	✓
M10	50	L2	60	10	✓	✓

12. USER INSTRUCTIONS

Required Software:

- Python 3.8 or higher
- Jupyter Notebook or Google Colab
- Libraries: pandas, matplotlib, numpy (all included in standard Anaconda/Colab environments)

How to Run:

1. Open the project notebook file (.ipynb) in Jupyter Notebook or upload it to Google Colab.

2. Run all cells from top to bottom using Cell > Run All (Jupyter) or Runtime > Run All (Colab).

3. No additional configuration or external data files are required. All data is defined within the notebook.

Expected Output:

- Two DataFrames showing the lab and module datasets.
- A compatibility table showing which labs are valid for each module.
- Two greedy assignment tables with an Unused Seats column.
- A bar chart comparing total unused seats between the two greedy algorithms.
- Hill-climbing result tables and improvement logs for each greedy starting point.
- Swap search result tables and improvement logs for each hill-climb result.
- A grouped bar chart comparing total unused seats across all six stages (Greedy, Hill Climb, Swap Search) for both algorithms.

13. SCREENSHOTS / OUTPUT EXAMPLES

	Lab ID	Name	Capacity	Software
0	L1	Advanced Lab A	100	[Python, Java, MATLAB]
1	L2	Coding Lab B	60	[Python, Java]
2	L3	Data Lab C	40	[Python, MATLAB]
3	L4	Legacy Lab D	30	[Java, MATLAB]

Figure 1: Labs Dataset

	Module Code	Students	Software Needed
0	M1	85	Python
1	M2	45	MATLAB
2	M3	25	Java
3	M4	55	Python
4	M5	35	MATLAB
5	M6	15	Java
6	M7	90	Java
7	M8	30	Python
8	M9	20	MATLAB
9	M10	50	Python

Figure 2 Modules Dataset

	Module Code	Students	Software Needed	Compatible Labs
0	M1	85	Python	[L1]
1	M2	45	MATLAB	[L1]
2	M3	25	Java	[L1, L2, L4]
3	M4	55	Python	[L1, L2]
4	M5	35	MATLAB	[L1, L3]

	Module Code	Students	Software Needed	Compatible Labs
5	M6	15	Java	[L1, L2, L4]
6	M7	90	Java	[L1]
7	M8	30	Python	[L1, L2, L3]
8	M9	20	MATLAB	[L1, L3, L4]
9	M10	50	Python	[L1, L2]

Figure 3 Compatibility mapping DataFrame

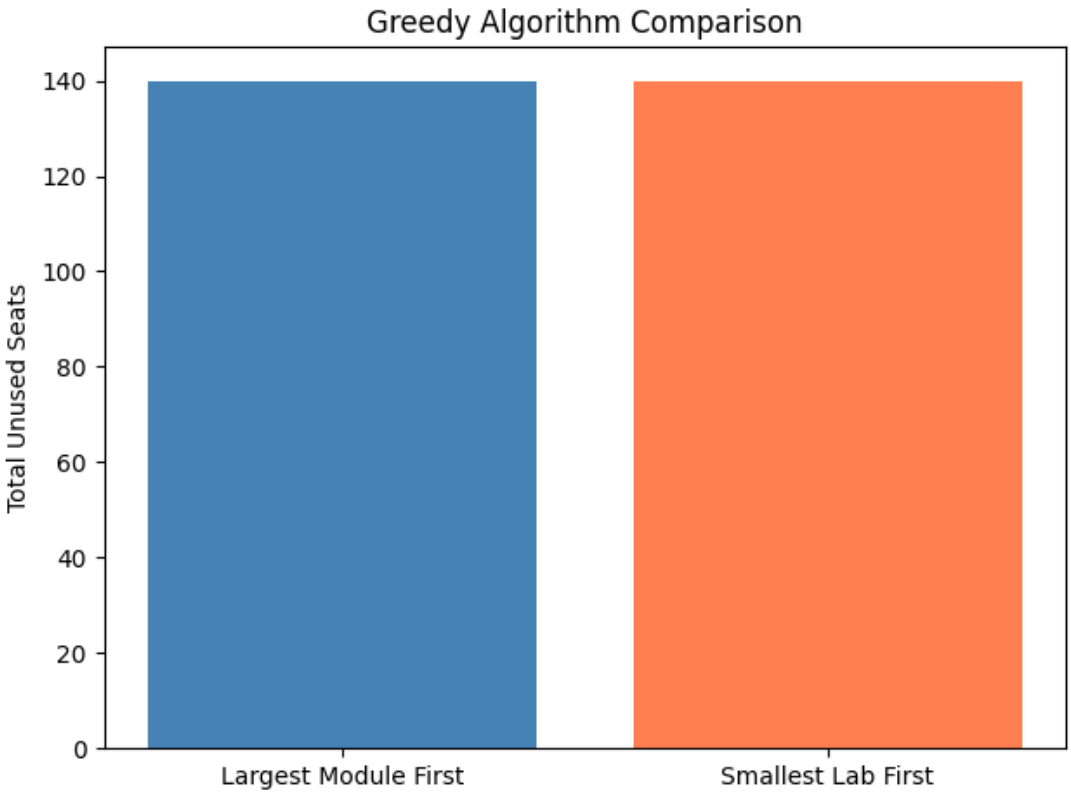

Figure 4 Initial Greedy Bar Chart Comparison

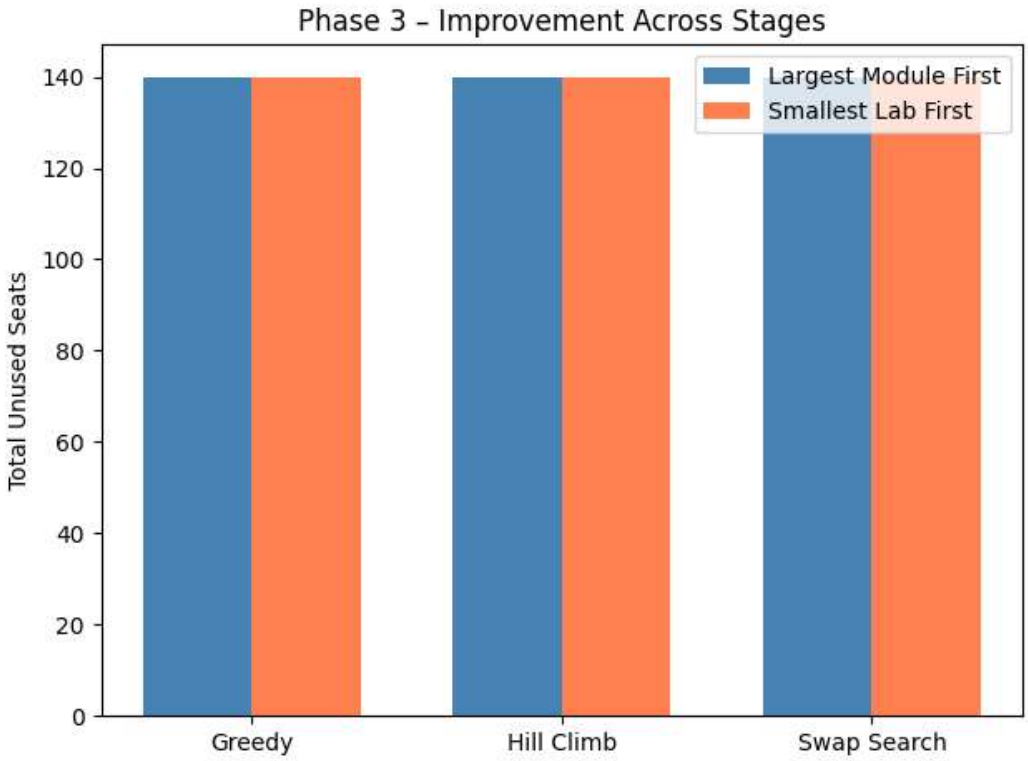

Figure 3 Final Grouped Bar Chart Comparison

14. REFLECTION ON GROUP WORK

The project was divided according to each member's assigned role. Kirone built the code architecture and greedy algorithms, focusing on how the functions were structured and how data flowed through the system. Shaista designed and refined the heuristic methods, deciding on the hill-climbing and swap search strategies and justifying the algorithmic choices. Tashmika was responsible for the mathematical model and system design, establishing the formal problem definition, decision variables and constraints. Josh evaluated the results, led the performance analysis and comparison, and was responsible for interpreting what the outcomes meant in the context of the optimization objective. Jeryn coordinated the overall report, ensured consistency across all sections, and handled quality checks on both the code output and written content.

The group used a Discord server as the main communication channel for discussions, updates, and decisions throughout the project. Code was developed and shared through a Google Colab notebook that all members could access and run. The report was written collaboratively through a shared Word document, where each member contributed their sections and could see changes made by others in real time. This setup meant that the code, communication, and writing were all kept in one place per

purpose, which made coordination straightforward. Decisions about the algorithms and results were discussed as a group to make sure everyone understood the full solution, not just their own section.

The biggest collaborative challenge was timing, sections that depended on the final code output, such as the results tables, screenshots, and performance analysis, could only be completed once the notebook was fully working. This created a bottleneck where some members had to wait before they could finalize their contributions. Keeping the report consistent in tone and formatting across five separately written sections in the shared Word document also required several rounds of review and editing. On the technical side, confirming that the greedy solution was already at a local optimum and that the improvement methods had nothing left to improve was an unexpected outcome that required the group to rethink how to frame and discuss the results meaningfully.

15. DECLARATION

We confirm that:

This work is our own.

All group members contributed as stated.

Signed:

- Jeryn Naidoo
- Shaista Naicker
- Kirone Gopaul
- Josh Goodwin
- Tashmika Pillay

Date: 19/05/26